

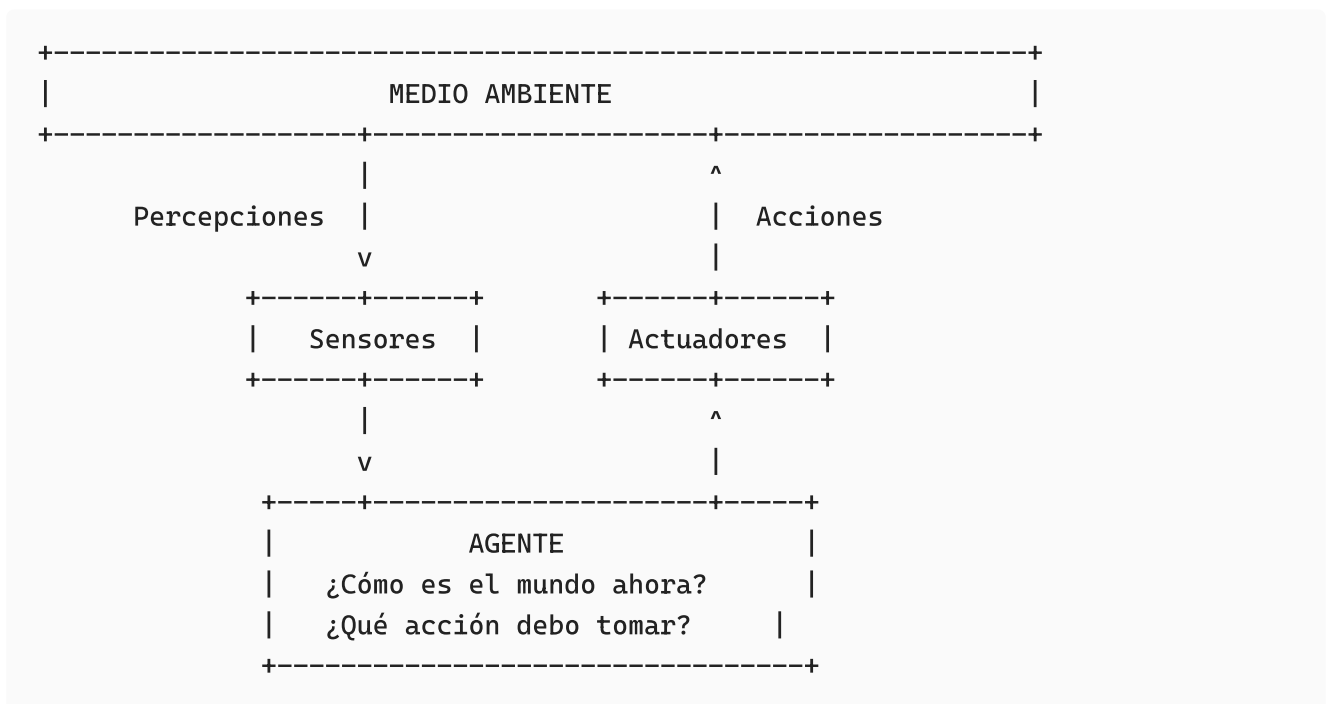
Apuntes SI T1-T5

TEMA 1: Introducción a los Sistemas Inteligentes

1.1 Concepto de Inteligencia Artificial

La Inteligencia Artificial (IA) se define formalmente como la disciplina científica centrada en la creación de programas informáticos que ejecutan operaciones comparables a las de la mente humana, destacando capacidades críticas como el aprendizaje autónomo o el razonamiento lógico.

Un agente inteligente se caracteriza por percibir un entorno complejo (físico o digital) mediante la adquisición de datos, interpretar dicha información (estructurada o no), razonar sobre los conocimientos derivados y seleccionar de manera autónoma las mejores acciones para cumplir una meta determinada.



1.2 Características Diferenciales de la IA

Frente al software algorítmico convencional, un sistema de IA se distingue por los siguientes ejes funcionales:

- **Procesos Heurísticos vs. Algorítmicos:** En lugar de seguir secuencias fijas de pasos matemáticos explícitos, los programas de IA emplean métodos heurísticos para guiar la búsqueda en espacios de soluciones complejos.

- **Separación de Información y Control:** A diferencia de la programación tradicional, donde las reglas lógicas y las estructuras de datos están integradas en el flujo del código, la IA simbólica mantiene la base de conocimientos estrictamente independiente del motor de inferencias.
- **Manejo de la Incertidumbre:** Capacidad de adaptación para tomar decisiones bajo entornos con datos incompletos, cambiantes o estocásticos.

1.3 Paradigmas de la IA: Simbólica vs. Subsimbólica

El desarrollo de la disciplina ha dado lugar a dos enfoques metodológicos complementarios:

Dimensión	IA Simbólica (Top-Down / Pulcros)	IA Subsimbólica (Bottom-Up / Desaliñados)
Representación	Conocimiento explícito estructurado mediante símbolos, reglas, hechos y ontologías.	Conocimiento implícito codificado numéricamente y aprendido a partir de datos.
Mecanismo	Inferencia lógica formal y separación estricta entre conocimiento y razonamiento.	Aprendizaje automático de patrones estadísticos y optimización de redes neuronales.
Explicabilidad	Alta (XAI): Permite realizar un rastreo paso a paso de la cadena de decisiones lógicas.	Baja (Caja Negra): Dificultad extrema para justificar formalmente una salida específica.
Limitaciones	Fragilidad ante inconsistencias y cuello de botella en la adquisición manual de conocimientos.	Alto coste computacional, riesgo de sesgos y nula trazabilidad deductiva.

Note

Tendencia Actual (IA Neuro-Simbólica): La convergencia moderna combina la robustez y estructuración de los enfoques basados en el conocimiento experto con la potencia de generalización de patrones procedentes del aprendizaje profundo.

TEMA 2: Algoritmos de Búsqueda en Espacios de Estados

2.1 Formalización de Problemas de Búsqueda

Los problemas prácticos se modelan mediante estructuras de grafos o árboles de expansión donde se definen los siguientes componentes:

- **Estado (n):** Representación de una configuración específica dentro del espacio del problema.
- **Frontera ($X(F)$ o F):** Conjunto dinámico que almacena todos los nodos que han sido generados por el sistema pero que aún no han sido explorados o evaluados.
- **Función de Evaluación ($f(n)$):** Evalúa la prioridad de un nodo en la frontera. Clásicamente se descompone como:

$$f(n) = g(n) + h(n)$$

- $g(n)$: Coste acumulado real del camino desde el nodo inicial hasta el nodo actual n .
- $h(n)$: Estimación heurística del coste restante desde el nodo n hasta la meta más cercana.

2.2 GUÍA METODOLÓGICA: Mecánica de Ejercicios de Búsqueda

Para resolver cualquier ejercicio práctico de expansión de árboles en exámenes, se debe seguir estrictamente este protocolo algorítmico sistemático:

PASO 1: Inicialización

1. Crea una lista para la **Frontera** y otra para los **Nodos Visitados**.
2. Introduce el nodo inicial en la frontera computando su valor de prioridad $f(n)$.

PASO 2: Extracción y Evaluación

1. Extrae el nodo prioritario de la frontera según la estrategia de búsqueda elegida (menor $f(n)$, orden de nivel o profundidad).
2. Si el nodo extraído coincide con la condición **Meta**, detén el algoritmo; el camino óptimo se ha encontrado con éxito.
3. Si no es meta, añádelo a la lista de **Visitados**.

PASO 3: Generación y Actualización de Frontera

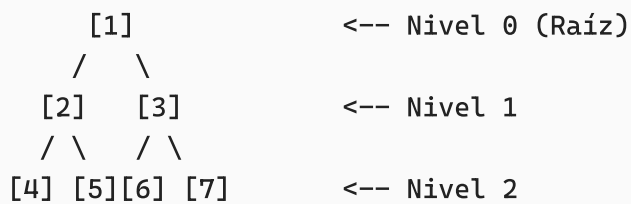
1. Expande el nodo actual identificando todos sus nodos hijos directamente accesibles.
2. Para cada hijo, calcula sus métricas de coste (g , h , f) según corresponda a la variante del algoritmo.
3. Añade los nuevos hijos a la frontera (si la estrategia implementa control de repetidos, ignora aquellos hijos que ya se encuentren en la lista de visitados con un coste igual o mayor).
4. Reordena la frontera de acuerdo con el criterio de prioridad del algoritmo. Repite desde el **Paso 2**.

2.3 Catálogo Analítico de Estrategias de Búsqueda

2.3.1 Búsqueda en Amplitud (Breadth-First Search - BFS)

- **Mecánica:** Explora de forma sistemática el grafo por niveles completos, avanzando de izquierda a derecha en el árbol de expansión.
- **Frontera:** Funciona como una cola FIFO (First-In, First-Out). Los nuevos nodos se insertan siempre al final.
- **Propiedades:** Es **Completa**. Es **Óptima** si y solo si todos los pasos del árbol poseen exactamente el mismo coste unitario.

Markdown



Este árbol representa el orden secuencial por niveles en el que se busca y expande cada nodo bajo la estrategia BFS.

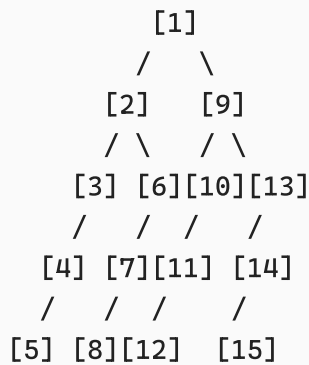
2.3.2 Búsqueda de Coste Uniforme (UCS / Algoritmo de Dijkstra)

- **Mecánica:** Variante de la búsqueda en amplitud para grafos con costes heterogéneos. Selecciona y explora prioritariamente aquel nodo de la frontera que minimice el coste real acumulado del camino: $f(n) = g(n)$.
- **Frontera:** Cola de prioridad ordenada ascendentemente por el valor de $g(n)$.
- **Propiedades:** Es **Completa** y **Óptima** independientemente de la variabilidad de los costes de los arcos. Si todos los arcos fuesen idénticos, su comportamiento equivale a una búsqueda en amplitud.

2.3.3 Búsqueda en Profundidad (Depth-First Search - DFS)

- **Mecánica:** Explora de manera exhaustiva cada camino lineal hasta alcanzar un nodo hoja antes de realizar marcha atrás (backtracking) para cambiar de camino.
- **Frontera:** Funciona como una pila LIFO (Last-In, First-Out). Los nodos más recientes se procesan primero.
- **Propiedades:** **No es Óptima**. Es **Completa únicamente en espacios de estados finitos**. En espacios infinitos, si el algoritmo entra en una rama infinita que no contiene la meta, se colgará y nunca hallará la solución.

Markdown



Este árbol representa el orden vertical exhaustivo en el que se busca y expande cada nodo bajo la estrategia DFS.

2.3.4 Búsqueda de Profundidad Limitada

- **Mecánica:** Resuelve el problema crítico de las ramas infinitas de DFS imponiendo un límite estricto de profundidad máxima l . Los nodos situados a una profundidad mayor son tratados directamente como nodos hoja.
- **Propiedades:** **No es Completa** (si la meta está a una profundidad mayor que l) y **No es Óptima**.

2.3.5 Búsqueda en Profundidad Iterativa (IDDFS)

- **Mecánica:** Ejecuta de forma sucesiva búsquedas en profundidad limitada, incrementando el límite de nivel ($l = 0, 1, 2, \dots$) de manera iterativa si no se halla la meta en la iteración previa.
- **Propiedades:** Consigue combinar la eficiencia espacial de memoria de la búsqueda en profundidad con las garantías de la búsqueda en amplitud: es **Completa** y **Óptima**.

2.3.6 Búsqueda Preferente por el Mejor (Avara / Greedy Best-First)

- **Mecánica:** Estrategia informada que utiliza únicamente la estimación del coste restante hacia la meta para guiar la exploración: $f(n) = h(n)$.
- **Propiedades:** **No es Óptima**. Es **Completa si y solo si el espacio de estados es estrictamente finito** y se implementa adicionalmente un control de nodos repetidos para evitar bucles de realimentación.

2.3.7 Búsqueda A^* (Búsqueda Informada)

- **Mecánica:** Algoritmo de referencia en IA que combina de manera equilibrada el coste real y el estimado: $f(n) = g(n) + h(n)$.
- **Garantía de Optimalidad:** El algoritmo A^* tiene la garantía absoluta de ser óptimo si su función heurística $h(n)$ es **consistente** (o monótona). Una heurística es consistente si

cumple formalmente la siguiente inecuación triangular para cualquier nodo n y cualquier sucesor n' generado por una acción de coste $c(n, a, n')$:

$$h(n) \leq c(n, a, n')$$

2.4 Algoritmos de Búsqueda Local (Optimización)

Se aplican cuando el camino para llegar a la solución no es relevante, sino que únicamente importa encontrar el estado óptimo final (maximizar o minimizar una función objetivo).

- **Cresta (Ridge):** Configuración del espacio de estados donde los valores heurísticos adyacentes son mejores que el actual, pero a los cuales no se puede acceder mediante transiciones simples o movimientos individuales en un único eje (los nodos hijos son peores, aunque existen nietos mejores).
- **Máximo Local:** Estado óptimo relativo donde todos los nodos hijos directamente adyacentes presentan valores peores o de menor rendimiento que el estado actual, bloqueando la progresión del algoritmo.
- **Meseta (Plateau):** Región plana del espacio de estados donde los nodos vecinos presentan el mismo valor heurístico exacto, impidiendo determinar una dirección de ascenso clara.
- **Mínimo Local:** Configuración inversa al máximo local; estado en el cual todos los nodos hijos son estrictamente mejores o de mayor valor, impidiendo continuar con el descenso en problemas de minimización.

TEMA 3: Representación del Conocimiento

3.1 Modelos Declarativos vs. Procedimentales

El conocimiento dentro de una arquitectura inteligente clásica se puede estructurar bajo dos enfoques de representación formal:

1. **Métodos Declarativos:** El conocimiento se expresa como una colección estática de hechos explícitos sobre el mundo. Las verdades del dominio se almacenan de forma independiente y resulta sencillo incorporar nuevos conocimientos sin alterar las estructuras lógicas ya existentes.
2. **Métodos Procedimentales:** El conocimiento se codifica directamente en forma de procedimientos operativos dinámicos (reglas de acción). Priorizan las capacidades inferenciales del sistema, permiten integrar de forma nativa el conocimiento heurístico de expertos y facilitan el uso de información procedimental.

3.2 Redes Semánticas y Marcos (Frames)

- **Redes Semánticas:** Representación gráfica del conocimiento lingüístico estructurada como un grafo con nodos (conceptos, entidades o categorías) unidos mediante aristas o arcos etiquetados unidireccionales que definen relaciones semánticas binarias. Las relaciones estructurales de referencia son `ES_UN` (Is-A, para taxonomías jerárquicas con herencia de propiedades) y `ES_PARTE_DE` (Part-Of, para descomposiciones estructurales de objetos).
- **Marcos (Frames):** Estructuras complejas destinadas a representar objetos mediante atributos organizados jerárquicamente. Se componen de una *Cabecera* (nombre de la clase) y múltiples *Slots* (ranuras de propiedades que pueden anidarse a cualquier nivel de profundidad). Los slots admiten la inserción de **Demons (Demonios)**: procedimientos de código procedimental latentes (`If_needed` , `If_added` , `If_removed`) que se disparan automáticamente ante modificaciones en los datos del marco, aportando un comportamiento dinámico al modelo declarativo.

3.3 Lógica Proposicional y Demostración de Teoremas

La lógica de proposiciones evalúa la veracidad de sentencias complejas construidas mediante conectivas lógicas ($\neg$, $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$) sobre variables de corte booleano.

- **Implicación Lógica ($\alpha \models \beta$):** Se cumple si y solo si en todos los modelos donde la sentencia α es verdadera, la sentencia β también es obligatoriamente verdadera ($M(\alpha) \subseteq M(\beta)$).
- **Teorema de Deducción:** Determinar que $\alpha \models \beta$ es matemáticamente equivalente a demostrar que la implicación condicional $\alpha \Rightarrow \beta$ constituye una tautología válida (cierta en todos sus modelos posibles).
- **Reducción al Absurdo (Refutación):** El razonamiento $\alpha \models \beta$ se verifica formalmente si se demuestra que la conjunción combinada ($\alpha \wedge \neg\beta$) es completamente insatisfacible (lo que da base al algoritmo de resolución).

3.4 Forma Normal Conjuntiva (CNF)

Para aplicar el algoritmo de resolución universal, cualquier sentencia lógica debe traducirse obligatoriamente a una Forma Normal Conjuntiva, caracterizada por ser una **conjunción de cláusulas**, donde cada cláusula es a su vez una **disyunción de literales**.

GUÍA PASO A PASO: Conversión de Sentencias Lógicas a CNF

Para transformar cualquier expresión lógica a cláusulas CNF en ejercicios prácticos, aplica estrictamente el siguiente orden algebraico:

- **PASO 1: Eliminación de Bicondicionales ($\Leftrightarrow$)** Sustituye cada ocurrencia del operador bicondicional empleando la regla de equivalencia:

razonamiento estructuradas bajo la sintaxis condicional IF <Premisa> THEN <Acción/Conclusión>).

- **Memoria Activa (MA):** Almacén dinámico que representa de manera fidedigna el estado actual del sistema en tiempo real. Contiene tanto los datos de entrada como los hechos intermedios deducidos y las hipótesis bajo investigación activa.
- **Motor de Inferencias (MI):** Componente de control encargado de ejecutar de manera secuencial el ciclo básico de operación del sistema.

4.2 El Ciclo Básico de Producción

El Motor de Inferencias opera iterativamente ejecutando las siguientes tres fases lógicas:

1. **Restricción:** Fase estática o dinámica que descarta a priori aquellas reglas de la BR que no poseen ninguna vinculación semántica con las variables registradas en la memoria activa, optimizando el rendimiento del sistema.
2. **Equiparación (Emparejamiento / Pattern Matching):** Proceso de comparación donde se contrastan las premisas de las reglas válidas frente a los hechos consolidados en la MA. El resultado es el **Conjunto Conflicto**, que agrupa a todas las reglas que cumplen sus condiciones y son aptas para ser ejecutadas en el ciclo actual.
3. **Resolución de Conflictos:** Algoritmo de decisión encargado de seleccionar una única regla del conjunto conflicto para su disparo inmediato. Los criterios de selección normativos incluyen:
 - *Orden alfabético/numérico:* Prioridad por la posición física de la regla en el archivo.
 - *Especificidad:* Se dispara la regla que valide un mayor número de condiciones en su premisa (procesa más información específica).
 - *Recencia:* Prioridad a las reglas que operen sobre los hechos añadidos más recientemente a la memoria.
 - *No duplicación:* Restricción estricta que prohíbe disparar exactamente la misma regla sobre los mismos hechos de datos más de una vez.

4.3 GUÍA PASO A PASO: Mecánica de Ejercicios del Tema 4 (Sistemas de Producción)

Para resolver con éxito los problemas prácticos de traza de Sistemas de Producción planteados en exámenes, se deben estructurar los datos bajo dos metodologías operativas diferentes:

Variante A: Inferencia Dirigida por Datos (Encadenamiento Progresivo)

- **Objetivo:** Partir de los hechos iniciales conocidos para deducir nuevas conclusiones e incorporarlas progresivamente a la memoria hasta verificar la meta.

[Hechos Iniciales] ----> [Filtro de Reglas Activas] ----> [Resolución de Conflictos] ----> [Nuevo Hecho en MA]

1. **Escribe el Estado Inicial de la Memoria Activa (M_0)** listando todos los datos iniciales confirmados como ciertos.
2. **Fase de Equiparación:** Identifica qué reglas de la Base de Reglas tienen sus condiciones (IF) completamente satisfechas por los hechos presentes en la MA en este ciclo. Anótalas en la columna de **Reglas Activadas**.
3. **Fase de Resolución de Conflictos:** Aplica las restricciones específicas indicadas en el enunciado (por ejemplo: orden físico de la base de reglas, recencia o especificidad) para elegir una única regla ganadora. Anótala en **Regla Ejecutada**.
4. **Fase de Acción:** Toma la conclusión (THEN) de la regla ejecutada e incorpórala a la MA, generando un nuevo estado de la memoria (M_{n+1}). Marca dicha regla para evitar su duplicación.
5. **Condición de Parada:** Evalúa si la hipótesis objetivo (H) ya forma parte de la MA como un hecho de datos consolidado. Si es así, finaliza el ejercicio indicando el número total de ciclos transcurridos. Si no, repite desde el paso 2.

Variante B: Inferencia Dirigida por Metas (Encadenamiento Regresivo)

- **Objetivo:** Partir de la meta final que se desea demostrar y retropropagar submetas lógicas hacia atrás buscando reglas que las concluyan, deteniéndose cuando las submetas se validen directamente con los hechos de la memoria.

[Meta Inicial ¿H?] ----> [Buscar Regla con Conclusión H] ----> [Establecer Antecedentes como Submetas] ----> [Validar con Datos]

1. **Fase de Retropropagación:** Coloca la meta principal en la pila de **Hipótesis de la Memoria Activa**.
2. Localiza todas las reglas de la BR cuya conclusión (THEN) satisfaga la hipótesis actual bajo examen.
3. Analiza las premisas (IF) de dichas reglas:
 - Si alguna premisa no se encuentra en la MA, se añade inmediatamente a la memoria en calidad de **Nueva Submeta (o subhipótesis)**, requiriendo su propia retropropagación hacia atrás.
 - Si todas las premisas de una regla coinciden directamente con hechos de datos verdaderos ya existentes en la MA, la regla pasa a ser **Directamente Ejecutable**.
4. **Fase de Ejecución en Cascada:** Comienza la resolución hacia adelante. Se disparan consecutivamente las reglas cuyas submetas han sido validadas, transformando las

hipótesis en hechos de datos reales demostrados uno a uno, hasta conseguir consolidar y verificar con éxito la hipótesis meta original.

4.4 Planteamiento de un Ejercicio Práctico Modelo (Resolución de Traza)

A continuación, se plantea un ejercicio práctico resuelto para mostrar detalladamente la mecánica de ejecución lógica de ambas variantes sobre un mismo conjunto conflicto.

Definición de la Base de Reglas y Hechos:

- **Base de Reglas (BR):**
 - R_1 : IF (X and Y) THEN Z
 - R_2 : IF (C and D) THEN G
 - R_3 : IF (E and V) THEN H
 - R_4 : IF (A and B) THEN C
 - R_5 : IF (F or G) THEN X
 - R_6 : IF (Z and B) THEN V
 - R_7 : IF (E and C) THEN F
- **Memoria Activa Inicial (M_0):** A, B, D, Y, E con la Hipótesis Objetivo entre paréntesis (H) .

SOLUCIÓN DEL ENTORNO A: Encadenamiento Progresivo (Dirigido por Datos)

Restricciones del motor de inferencias: Dirigido por datos, se activan todas las reglas que emparejen, búsqueda en profundidad ejecutando la primera regla activa de la base de reglas (según su orden físico) que sea más reciente, sin duplicados, y termina al demostrar el hecho H .

- **Ciclo 1:**
 - **Memoria Activa (M_0):** <A, (H), B, D, E, Y>
 - **Reglas Activadas:** R_4 (requiere A y B, presentes en M_0).
 - **Regla Ejecutada:** R_4 .
 - **Resultado:** Se añade la conclusión C a la memoria.
 - **Nueva Memoria (M_1):** <A, (H), B, C, D, E, Y>
- **Ciclo 2:**
 - **Memoria Activa (M_1):** <A, (H), B, C, D, E, Y>
 - **Reglas Activadas:** R_2 (requiere C y D), R_7 (requiere E y C).
 - **Regla Ejecutada:** R_2 (por orden físico de la BR al activarse en este ciclo).

- **Resultado:** Se añade la conclusión G a la memoria.
- **Nueva Memoria (M_2):** $\langle A, (H), B, C, D, E, G, Y \rangle$
- **Ciclo 3:**
 - **Memoria Activa (M_2):** $\langle A, (H), B, C, D, E, G, Y \rangle$
 - **Reglas Activadas:** R_5 (requiere F o G , G está presente), R_7 (pendiente del ciclo anterior).
 - **Regla Ejecutada:** R_5 (por criterio de recencia, se activó en este ciclo frente a R_7 que venía del anterior).
 - **Resultado:** Se añade la conclusión X a la memoria.
 - **Nueva Memoria (M_3):** $\langle A, (H), B, C, D, E, G, X, Y \rangle$
- **Ciclo 4:**
 - **Memoria Activa (M_3):** $\langle A, (H), B, C, D, E, G, X, Y \rangle$
 - **Reglas Activadas:** R_1 (requiere X y Y , ambos presentes), R_7 (pendiente).
 - **Regla Ejecutada:** R_1 (por criterio de recencia de activación).
 - **Resultado:** Se añade la conclusión Z a la memoria.
 - **Nueva Memoria (M_4):** $\langle A, (H), B, C, D, E, G, X, Y, Z \rangle$
- **Ciclo 5:**
 - **Memoria Activa (M_4):** $\langle A, (H), B, C, D, E, G, X, Y, Z \rangle$
 - **Reglas Activadas:** R_6 (requiere Z y B), R_7 (pendiente).
 - **Regla Ejecutada:** R_6 (por recencia).
 - **Resultado:** Se añade la conclusión V a la memoria.
 - **Nueva Memoria (M_5):** $\langle A, (H), B, C, D, E, G, V, X, Y, Z \rangle$
- **Ciclo 6:**
 - **Memoria Activa (M_5):** $\langle A, (H), B, C, D, E, G, V, X, Y, Z \rangle$
 - **Reglas Activadas:** R_3 (requiere E y V), R_7 (pendiente).
 - **Regla Ejecutada:** R_3 (por recencia).
 - **Resultado:** Se añade la conclusión H a la memoria.
 - **Nueva Memoria (M_6):** $\langle A, B, C, D, E, G, H, V, X, Y, Z \rangle$
 - **Condición de Parada:** $H \in M_6$. El proceso termina con éxito tras **6 ciclos**.

Markdown

```
A, B ----> [R4] ----> C
C, D ----> [R2] ----> G ----> [R5] ----> X
X, Y ----> [R1] ----> Z
Z, B ----> [R6] ----> V
V, E ----> [R3] ----> H (META LOGRADA)
```

Este diagrama representa de forma secuencial el circuito inferencial progresivo completo generado por el encadenamiento de las reglas disparadas en este ejercicio.

SOLUCIÓN DEL ENTORNO B: Encadenamiento Regresivo (Dirigido por Metas)

Restricciones del motor de inferencias: Dirigido por objetivos, activación global, búsqueda en anchura exhaustiva, no duplicados, finaliza al consolidar H .

Fase de Retropropagación (Establecimiento de Submetas)

- **Ciclo 1:** Meta inicial: H . Buscamos reglas que concluyan H . Encontramos R_3 (IF E and V). Como el dato E ya es un hecho cierto en la MA, pero V no lo es ($V \notin M_0$), se genera la **submeta** (V) .
 - *Memoria Activa:* $\langle A, (H), (V), B, D, E, Y \rangle$
- **Ciclo 2:** Investigamos la submeta V . Buscamos reglas que concluyan V . Encontramos R_6 (IF Z and B). El hecho B es conocido, pero Z no. Se genera la **submeta** (Z) .
 - *Memoria Activa:* $\langle A, (H), (V), (Z), B, D, E, Y \rangle$
- **Ciclo 3:** Investigamos la submeta Z . Buscamos reglas que concluyan Z . Encontramos R_1 (IF X and Y). El hecho Y es conocido, pero X no. Se genera la **submeta** (X) .
 - *Memoria Activa:* $\langle A, (H), (V), (X), (Z), B, D, E, Y \rangle$
- **Ciclo 4:** Investigamos la submeta X . Buscamos reglas que concluyan X . Encontramos R_5 (IF F or G). Como es un operador de disyunción OR, se generan en anchura las **submetas** (F) y (G) .
 - *Memoria Activa:* $\langle A, (F), (G), (H), (V), (X), (Z), B, D, E, Y \rangle$
- **Ciclo 5:** Investigamos las submetas F y G en paralelo.
 - Para G : Encontramos R_2 (IF C and D). El hecho D es conocido, pero C no.
 - Para F : Encontramos R_7 (IF E and C). El hecho E es conocido, pero C no.
 - Ambas ramas convergen solicitando la misma **submeta común** (C) .
 - *Memoria Activa:* $\langle A, (C), (F), (G), (H), (V), (X), (Z), B, D, E, Y \rangle$

Fase de Ejecución en Cascada (Disparo de Reglas)

- **Ciclo 6:** Investigamos la submeta C . Buscamos reglas que concluyan C . Encontramos R_4 (IF A and B). Como tanto A como B son hechos reales iniciales, se ejecuta R_4 y la hipótesis C se consolida como hecho real de datos.
 - *Memoria Activa:* $\langle A, (F), (G), (H), (V), (X), (Z), B, C, D, E, Y \rangle$
- **Ciclo 7:** Al ser C un hecho, las premisas de R_2 y R_7 pasan a estar verificadas por datos reales. Se ejecutan en paralelo, consolidando F y G como hechos ciertos.
 - *Memoria Activa:* $\langle A, (H), (V), (X), (Z), B, C, D, E, F, G, Y \rangle$

- **Ciclo 8:** Se valida la premisa de R_5 (F or G). Se ejecuta R_5 transformando la hipótesis X en hecho.
 - *Memoria Activa:* $\langle A, (H), (V), (Z), B, C, D, E, F, G, X, Y \rangle$
- **Ciclo 9:** Se valida la premisa de R_1 (X and Y). Se ejecuta R_1 , consolidando la hipótesis Z como hecho.
 - *Memoria Activa:* $\langle A, (H), (V), B, C, D, E, F, G, X, Y, Z \rangle$
- **Ciclo 10:** Se valida la premisa de R_6 (Z and B). Se ejecuta R_6 , consolidando la hipótesis V como hecho verídico.
 - *Memoria Activa:* $\langle A, (H), B, C, D, E, F, G, V, X, Y, Z \rangle$
- **Ciclo 11:** Se valida la regla inicial R_3 (E and V). Se ejecuta R_3 , consolidando la meta principal H como hecho definitivo de la memoria.
 - *Memoria Activa:* $M11 = \langle A, B, D, Y, E, H, V, Z, X, F, G, C \rangle$
 - El proceso termina de manera satisfactoria tras **11 ciclos** del motor de inferencias.

TEMA 5: Sistemas de Planificación Clásica

5.1 Conceptos Fundamentales

La Planificación Clásica consiste en la generación automática de una secuencia ordenada de acciones lógicas (denominada **Plan**) que permite a un agente transitar desde un *Estado Inicial* hasta un *Estado Objetivo* o meta específica. Se modela sobre entornos ideales bajo supuestos restrictivos: discretos, deterministas, estáticos y totalmente observables.

- **Búsqueda vs. Planificación:** En los problemas de búsqueda estándar, el espacio de estados y sus transiciones son completamente explícitos, limitándose el algoritmo a calcular rutas lógicas sobre un mapa predefinido. En cambio, en planificación el entorno se describe mediante lenguajes simbólicos estructurados; las acciones contienen precondiciones operativas y efectos explícitos que modifican de manera dinámica las variables del entorno, permitiendo al agente modelar las dinámicas internas del mundo.

5.2 El Modelo STRIPS

STRIPS (Stanford Research Institute Problem Solver) representa el estándar clásico de planificación fundamentado en tres componentes formalizados con lógica de literales:

1. **Estado:** Representado como un conjunto de literales lógicos o variables booleanas indexadas en un instante del tiempo.
2. **Objetivo (Meta):** Conjunto específico de condiciones lógicas o literales que se requiere que sean verdaderas de manera simultánea en el estado final.
3. **Operadores (Acciones):** Estructuras operativas compuestas por:

- **Precondiciones (Precon):** Literales lógicos obligatorios que deben cumplirse en el estado actual del mundo para que la acción pueda ser ejecutada válidamente por el agente.
- **Efectos / Postcondiciones (Postcon):** Modificaciones atómicas inducidas en el entorno tras el disparo de la acción. Se dividen formalmente en la **Lista de Añadir (Add List)** (literales que pasan a ser verdaderos) y la **Lista de Eliminar (Delete List)** (literales que pasan a ser falsos).

5.3 GUÍA PASO A PASO: Mecánica de Ejercicios de Planificación

Para resolver de forma sistemática los ejercicios de traza y modelado en planificación clásica, aplica el siguiente protocolo metodológico estructurado:

PASO 1: Formalización de Variables y Acciones

1. Define el vector de variables de estado con valores booleanos que describen las propiedades del entorno del problema.
2. Modela cada acción disponible especificando de forma nítida sus precondiciones obligatorias y sus efectos de adición/eliminación de variables.

PASO 2: Inicialización del Estado

1. Escribe de manera explícita la configuración de literales que componen el **Estado Inicial** (S_0).

PASO 3: Traza de Ejecución hacia Adelante (Forward Chaining)

1. Analiza el estado actual S_n . Identifica qué acciones de la base de operadores tienen sus **Precondiciones** totalmente satisfechas por los valores lógicos del estado actual.
2. Selecciona la acción que corresponda al plan bajo análisis.
3. Computa el nuevo **Estado Resultante** (S_{n+1}) aplicando estrictamente las postcondiciones del operador:
 - Modifica a **Verdadero** aquellas variables indicadas en los efectos de adición.
 - Modifica a **Falso** las variables presentes en la lista de eliminación.
 - Mantén inalteradas todas las demás variables del entorno (por el Principio de Inercia o *Frame Problem*).

PASO 4: Verificación de la Meta

1. Contrasta si el estado consolidado S_{n+1} satisface de forma íntegra las condiciones fijadas en el **Estado Objetivo**. Si se cumple, el plan es formalmente **Válido y Correcto**.

5.4 Planteamiento de un Ejercicio Práctico Modelo (Validación de Plan)

A continuación, se plantea un ejercicio modelo clásico enfocado en la resolución del problema lógico de rellenar una botella de agua en una fuente.

1. Formalización de Variables de Estado:

- E_1 : La botella está en la fuente
- E_2 : El agente está frente a la fuente
- E_3 : El agente tiene la botella
- E_4 : La fuente funciona
- E_5 : La botella está llena

2. Definición del Catálogo de Acciones (Operadores):

- **Moverse a la fuente**
 - *Precondiciones*: $E_2 = \text{FALSO}$
 - *Efectos (Postcondiciones)*: $E_2 = \text{VERDADERO}$
- **Colocar la botella**
 - *Precondiciones*: $E_3 = \text{VERDADERO} \wedge E_2 = \text{VERDADERO} \wedge E_1 = \text{FALSO}$
 - *Efectos (Postcondiciones)*: $E_3 = \text{FALSO} \wedge E_1 = \text{VERDADERO}$
- **Activar la fuente**
 - *Precondiciones*: $E_2 = \text{VERDADERO} \wedge E_1 = \text{VERDADERO} \wedge E_4 = \text{VERDADERO}$
 - *Efectos (Postcondiciones)*: $E_5 = \text{VERDADERO}$
- **Recoger la botella**
 - *Precondiciones*: $E_2 = \text{VERDADERO} \wedge E_1 = \text{VERDADERO} \wedge E_3 = \text{FALSO}$
 - *Efectos (Postcondiciones)*: $E_3 = \text{VERDADERO} \wedge E_1 = \text{FALSO}$

3. Especificación de Estados Críticos:

- **Estado Inicial (S_0)**: $[E_1 = 0, E_2 = 0, E_3 = 1, E_4 = 1, E_5 = 0]$
- **Estado Objetivo (Meta)**: $[E_3 = 1, E_5 = 1]$

4. Traza Completa de Ejecución del Plan:

- **Estado Inicial S_0** : $[0, 0, 1, 1, 0]$
- **Acción 1**: Moverse a la fuente (Válida, pues cumple su precondición: $E_2 = 0$).
 - *Efecto*: Cambia $E_2 \rightarrow 1$.
 - *Estado Resultante S_1* : $[0, 1, 1, 1, 0]$
- **Estado Actual S_1** : $[0, 1, 1, 1, 0]$

- **Acción 2:** Colocar la botella (Válida, valida sus precondiciones: $E_3 = 1, E_2 = 1, E_1 = 0$).
 - *Efecto:* Cambia $E_3 \rightarrow 0, E_1 \rightarrow 1$.
 - *Estado Resultante S_2 :* $[1, 1, 0, 1, 0]$
- **Estado Actual S_2 :** $[1, 1, 0, 1, 0]$
- **Acción 3:** Activar la fuente (Válida, valida sus precondiciones: $E_2 = 1, E_1 = 1, E_4 = 1$).
 - *Efecto:* Cambia $E_5 \rightarrow 1$.
 - *Estado Resultante S_3 :* $[1, 1, 0, 1, 1]$
- **Estado Actual S_3 :** $[1, 1, 0, 1, 1]$
- **Acción 4:** Recoger la botella (Válida, valida sus precondiciones: $E_2 = 1, E_1 = 1, E_3 = 0$).
 - *Efecto:* Cambia $E_3 \rightarrow 1, E_1 \rightarrow 0$.
 - *Estado Resultante S_4 :* $[0, 1, 1, 1, 1]$
- **Evaluación Final:** El estado final consolidado S_4 posee las propiedades $E_3 = 1$ (El agente tiene la botella) y $E_5 = 1$ (La botella está llena). Las condiciones del **Estado Objetivo** se satisfacen por completo; por consiguiente, el plan analizado es plenamente **Válido y Correcto**.

ANEXO PRÁCTICO: Ejercicios de Examen Resueltos

Ejercicio 1: Traza Completa de un Sistema de Producción Reactivo

Se plantea el siguiente escenario práctico de evaluación para mostrar paso a paso la mecánica interna de orden de activación de un motor de reglas sobre una población de datos.

Inicialización de Estructuras Básicas:

- **Base de Reglas (BR):**
 - R_1 : IF $M(1) \vee M(2) \vee M(3)$ THEN $I(1) \vee I(2)$ (Identifica regla de emparejamiento indeterminado).
 - R_2 : IF $\neg I(2)$ THEN $\neg M(2) \wedge M(1)$ (Consistencia de asignación lógica).
 - R_3 : IF $I(1) \vee \neg I(2)$ THEN $M(2) \wedge M(3)$ (Identifica estados de conflicto).
- **Población de Hipótesis / Individuos en el espacio muestral:** Se definen cuatro individuos candidatos a solución mapeados en la memoria: i_1, i_2, i_3, i_4 .
- **Tabla de Verdad Estática de los Individuos (Matriz de Datos Inicial):**
 - $i_1 = [M(1) = 1, M(2) = 0, M(3) = 0, I(1) = 0, I(2) = 0]$
 - $i_2 = [M(1) = 1, M(2) = 0, M(3) = 0, I(1) = 0, I(2) = 1]$
 - $i_3 = [M(1) = 1, M(2) = 0, M(3) = 0, I(1) = 1, I(2) = 0]$
 - $i_4 = [M(1) = 1, M(2) = 0, M(3) = 0, I(1) = 1, I(2) = 1]$

Computación y Traza de Probabilidades:

El análisis heurístico computa los pesos de las funciones de evaluación asignadas a cada individuo del espacio mapeado:

- $P(i_1) = 0.3 \rightarrow \neg I(1) \wedge \neg I(2)$
- $P(i_2) = 0.2 \rightarrow \neg I(1) \wedge I(2)$
- $P(i_3) = 0.1 \rightarrow I(1) \wedge \neg I(2)$
- $P(i_4) = 0.4 \rightarrow I(1) \wedge I(2) \rightarrow$ **Individuo Más Probable / Óptimo Heurístico.**

Ejercicio 2: Resolución Práctica mediante el Teorema de Bayes (Cálculo Clínico)

Se plantea un problema práctico clásico de diagnóstico médico para ilustrar la aplicación del teorema inferencial de Bayes bajo incertidumbre sintomática.

Formalización de la Base de Hechos:

- **Variables de Manifestación Médica (Síntomas):**
 - $m(1)$: Fiebre
 - $m(2)$: Diarrea
 - $m(3)$: Tos
- **Variables de Diagnóstico (Patologías):**
 - $i(1)$: Gripe
 - $i(2)$: Gastroenteritis

Identificación del Estado del Paciente y Reglas de Diagnóstico:

- Hechos Iniciales Confirmados del Paciente (Manifestación clínica observada):
 - El paciente presenta la configuración de síntomas: $P_2 = m(1) \wedge \neg m(2) \wedge \neg m(3)$ (Fiebre en ausencia de diarrea y tos).
 - Esto consolida la variable de observación de evidencia m_6 en la base de hechos.
- El análisis combinatorio de las restricciones lógicas del dominio descarta los casos inconsistentes, dictaminando que únicamente los individuos i_3 e i_4 resultan matemáticamente compatibles bajo la evidencia observada de m_6 . Por consiguiente, se aplica la probabilidad condicionada sobre estas dos ramas.

Desarrollo Matemático del Teorema de Bayes:

Para determinar con exactitud la probabilidad de que el paciente sufra la patología asociada al individuo i_3 dada la evidencia observada de la manifestación m_6 , se plantea formalmente la ecuación general de Bayes:

$$P(i_3/m_6) = \frac{P(m_6/i_3) \cdot P(i_3)}{\sum_j P(m_6/i_j) \cdot P(i_j)}$$

Sustituyendo los valores numéricos por los sumandos de los caminos válidos exclusivos del espacio muestral (i_3 e i_4), dado que para el resto de combinaciones la probabilidad de ocurrencia bajo el síntoma es nula:

$$P(i_3/m_6) = \frac{P(m_6/i_3) \cdot P(i_3)}{P(m_6/i_1) \cdot P(i_1) + P(m_6/i_2) \cdot P(i_2) + P(m_6/i_3) \cdot P(i_3) + P(m_6/i_4) \cdot P(i_4)}$$

$$P(i_3/m_6) = \frac{0.3 \cdot 0.2}{(0.3 \cdot 0.2) + (0.1 \cdot 0.05)} = \frac{0.06}{0.06 + 0.005} = \frac{0.06}{0.065} = \mathbf{0.923}$$

Conclusión del Diagnóstico:

- La probabilidad final de la hipótesis diagnóstica del individuo i_3 bajo la evidencia clínica evaluada es del **92.3%** ($P(i_3/m_6) = 0.923$).
- Por complemento directo del espacio muestral restringido válido, la probabilidad de la hipótesis del individuo i_4 es de tan solo el **4%** ($P(i_4/m_6) = 0.04$).